

DATAIKU DSS

Guide Expert — Compétences & Fonctionnalités

Fiche de référence expert — Prijanth Seevaratnam

Mai 2026

1. Vue d'ensemble

Dataiku DSS (Data Science Studio) est une plateforme collaborative de data science et de machine learning. Elle permet à des profils variés (data scientists, data engineers, analystes, développeurs) de collaborer sur des projets de données de bout en bout : de l'ingestion à la mise en production de modèles.

Positionnement marché

- Concurrent direct : Alteryx, Knime, Azure ML Studio, SAS Viya, Palantir Foundry
- Points différenciants : interface visuelle + code natif, collaboration multi-profils, MLOps intégré
- Clients types : grandes entreprises (banques, assurances, retail, industrie)
- Modèle de licence : online (cloud) et on-premise, licences par utilisateur ou par nœud

Cas d'usage principaux

- Préparation et transformation de données à grande échelle
- Développement et déploiement de modèles ML (prédiction, scoring, clustering)
- Automatisation de pipelines data (batch et temps réel)
- Reporting analytique et tableaux de bord via webapps
- Feature store et gouvernance des modèles
- Orchestration de workflows complexes multi-systèmes

2. Architecture technique

Composants principaux

DSS Server (backend Java)

- Serveur central qui orchestre tous les traitements
- Stocke les métadonnées, la configuration et les résultats
- Expose une API REST complète pour l'automatisation externe

Engines de calcul (back-ends)

- In-memory DSS (Python/Pandas) : par défaut, limité à la RAM du serveur
- In-database : pousse les calculs vers la base source (SQL, Snowflake, BigQuery, Redshift...)
- Spark : distribué, intégration native Spark 2/3, PySpark / SparkR / SparkSQL
- Hadoop / Hive / Impala : sur cluster HDFS, via YARN
- Kubernetes : exécution de containers Docker pour l'isolation des workloads

Storage

- Managed datasets : stockés sur le filesystem du serveur DSS
- External datasets : pointeurs vers des sources externes (S3, ADLS, GCS, HDFS, JDBC...)
- Feature store : couche de partage de features entre projets

Déploiement et scaling

- Installation sur Linux (RHEL, Debian, Ubuntu) — pas de support Windows natif
- Architecture single-node (dev/test) ou multi-node avec clusters Spark/K8s
- API Node : nœud de déploiement séparé pour servir les modèles en production
- Automation Node : nœud dédié à l'exécution des scénarios automatiques
- Design Node : nœud de développement interactif
- Séparation recommandée design / automation / API en prod

3. Concepts fondamentaux

Objets clés DSS

Project

- Unité organisationnelle principale — contient tous les objets d'un cas d'usage
- Isolation des permissions : un projet = un périmètre de sécurité
- Versioning via Git intégré (commit, push, pull, merge sur un projet entier)

Dataset

- Représentation d'une source de données (table SQL, fichier CSV, S3, API...)
- Chaque dataset a un schéma, des statistiques et une configuration de stockage
- Types : Filesystem (CSV, Parquet, ORC...), SQL (PostgreSQL, MySQL, Oracle...), Cloud (S3, GCS, ADLS), NoSQL (MongoDB, Elasticsearch, Cassandra), Streaming (Kafka, Kinesis)
- Managed vs External : managed = DSS gère le stockage, external = la source est externe

Recipe

- Transformation entre datasets — c'est le cœur du flow de traitement
- Deux familles : Visual (no-code/low-code) et Code (Python, R, SQL, Spark...)
- Chaque recette est reproductible, versionnable et schedulable

Flow

- Représentation graphique DAG (Directed Acyclic Graph) des datasets et recettes
- Permet de visualiser les dépendances et l'impact d'un changement
- Recalcul partiel ou total (bouton "Build")
- Mécanisme de cache : si les inputs n'ont pas changé, on ne recalcule pas

Job

- Instance d'exécution d'un build de dataset ou de plusieurs datasets
- Log en temps réel, diagnostics d'erreur, historique des exécutions
- Statuts : Running, Done, Failed, Aborted

4. Recettes — Visual & Code

Recettes visuelles (Visual Recipes)

- Prepare : nettoyage et transformation colonne par colonne (200+ processeurs)
- Join : jointures entre datasets (inner, left, right, full, cross, fuzzy join)
- Group/Aggregate : agrégations (sum, count, avg, min, max, first, last, collect, percentile...)
- Window : fonctions fenêtre (lag, lead, rank, row_number, cumulative sum/avg)
- Stack : empilement vertical de datasets (UNION)
- Split : découpage d'un dataset en plusieurs selon une condition ou un ratio
- Sampling : échantillonnage (aléatoire, stratifié, top N, premier/dernier N%)
- Sort : tri sur une ou plusieurs colonnes
- Distinct : dédoublonnage
- Top N : filtrage des N premières lignes selon un critère
- Pivot : transformation lignes en colonnes
- Unpivot : transformation colonnes en lignes
- Sync : copie d'un dataset vers un autre format/stockage
- Download : téléchargement de données depuis une URL
- Upload : chargement de fichiers externes
- Export : export vers un système externe

Recettes Code

- Python : Pandas, NumPy, Scikit-learn, etc. — accès aux datasets via l'API DSS
- R : intégration tidyverse, data.table — accès aux datasets via library(dataiku)
- SQL : exécution de requêtes SQL dans la base cible (in-database)
- Scala : pour les workloads Spark avancés
- PySpark : Python sur Spark distribué
- SparkR : R sur Spark
- SparkSQL : SQL sur Spark
- Hive : requêtes HiveQL sur cluster Hadoop
- Impala : requêtes SQL sur HDFS via Impala
- Pig Latin : transformations Hadoop bas niveau
- Shell : scripts bash pour des opérations système
- Plugin recipes : recettes packagées via le système de plugins

Recette Prepare — détail processeurs clés

- Rename / Delete columns : gestion du schéma
- Filter rows : filtrage avec conditions (égalité, regex, range, NULL)
- Formula : colonnes calculées (DSS formula language, très proche Excel)
- Geopoint : extraction lat/lon, géocodage
- Date parsing : extraction year, month, day, quarter, diff entre dates
- Text processing : split, concat, trim, upper/lower, replace, regex extract
- Type casting : conversion de types (string ! int, date, etc.)
- Unfold / Fold : manipulation de listes/arrays
- Lookup : enrichissement par jointure simplifiée
- Flag / Category : encodage conditionnel
- Statistical functions : z-score, imputation de valeurs manquantes

5. Machine Learning & MLOps

Visual ML (AutoML)

Interface point-and-click pour créer des modèles de prédiction, classification ou clustering sans écrire de code.

- Supervised : Classification binaire, multiclass, Régression
- Unsupervised : Clustering (K-Means, DBSCAN, Gaussien)
- Anomaly detection (Isolation Forest, LOF)
- Computer vision, NLP (via deep learning — avec LLM extension)
- Sélection automatique d'algorithmes : RF, XGBoost, LightGBM, Logistic Reg., SVM, Neural Net...
- Hyperparameter tuning : grid search, random search, Bayesian optimization
- Feature engineering automatique : interactions, encodage catégoriel, gestion des NaN

Évaluation et comparaison de modèles

- Métriques classif : AUC-ROC, accuracy, F1, precision, recall, log-loss, calibration
- Métriques régression : RMSE, MAE, MAPE, R^2
- Session de comparaison : benchmark entre plusieurs runs avec hyperparamètres différents
- Variables d'importance (feature importance), SHAP values pour l'explicabilité
- Matrice de confusion, courbe ROC, lift/gain charts
- Backtesting time-based pour les modèles temporels

Déploiement et MLOps

- Model deployment sur API Node : REST endpoint pour scoring temps réel
- Batch scoring : dataset d'entrée ! dataset de sortie via recette de scoring
- Model lifecycle : Active, Archived, Champion/Challenger
- A/B testing entre deux versions de modèle
- Model monitoring : détection de dérive (data drift, concept drift)
- Retraining automatique via scénarios
- MLflow tracking intégré (métriques, artefacts, modèles)
- Export du modèle : pickle, PMML, ONNX
- Model documentation : génération automatique de rapport PDF

Feature Store

- Partage de features entre projets DSS
- Calcul des features en amont, réutilisation downstream
- Point-in-time correct retrieval pour éviter le data leakage
- Gestion de la fraîcheur et versioning des features

6. API Designer & Webapps

API Designer

- Création d'API REST sans code ou via Python custom
- Types d'endpoints : scoring (modèle), lookup (dataset), custom (Python)
- Authentification : API keys, JWT
- Déploiement sur API Node avec load balancing
- Monitoring des appels : latence, volume, erreurs
- Rate limiting et quotas configurables
- Swagger/OpenAPI documentation auto-générée

Webapps

- Bokeh : graphiques Python interactifs dans le navigateur
- Dash (Plotly) : dashboards Python réactifs
- Shiny : dashboards R interactifs
- HTML/JS : pages web full custom intégrées dans DSS
- Accès aux datasets DSS depuis les webapps via l'API Python/JS
- Authentification DSS héritée dans les webapps
- Déploiement sur Data Science Studio et/ou URL publique

7. Automatisation — Scenarios

Scénarios

Les scénarios permettent d'automatiser des workflows complets : rebuild de datasets, retraining de modèles, envoi de notifications, exécution de code personnalisé.

Déclencheurs (Triggers)

- Temporel : cron-like (ex: tous les jours à 6h, chaque lundi à 8h)
- Manuel : déclenchement à la demande via UI ou API
- Détection de fichier : dès qu'un nouveau fichier arrive sur S3, HDFS, etc.
- Résultat de scénario : chaînage de scénarios
- Signal SQL : quand une requête SQL renvoie un résultat > 0
- Dataset modified : quand un dataset upstream a été modifié
- Webhook : appel HTTP entrant depuis un système externe

Étapes (Steps)

- Build dataset(s) : recalcul d'un ou plusieurs datasets
- Train model : retraining automatique d'un modèle
- Deploy model : déploiement d'une nouvelle version
- Run scenario : appel d'un autre scénario
- Python script step : code Python arbitraire
- Send message : email, Slack, Teams, webhook sortant
- Set variables : mise à jour de variables DSS
- Export dataset : vers un système externe
- If/else : branchement conditionnel
- Abort if : arrêt conditionnel du scénario
- Compute metrics : calcul de métriques sur un dataset
- Checks : assertions sur les métriques (ex: completeness > 95%)

Monitoring & Alertes

- Métriques de dataset : completeness, distinctness, stats descriptives
- Checks : seuils d'alerte (green/yellow/red) sur les métriques
- Dashboards de monitoring : vue agrégée des checks sur tous les datasets
- Alertes email / Slack en cas d'échec de scénario ou de check

8. Plugins & Extensions

Plugin Hub

- Bibliothèque de +100 plugins officiels et communautaires
- Exemples : Geocoder, Fuzzy Join, Excel Export, SMTP Sender, Slack, Jira, SAP...
- Installation en 1 clic depuis l'interface DSS Admin
- Chaque plugin peut ajouter des recettes, connecteurs, processeurs Prepare, macros

Développement de plugins custom

- Structure : plugin.json + dossier de code Python/Java
- Types d'extensions possibles : recipe, connector, processor, webapp, macro, format
- Développement en Python (le plus courant) ou Java
- Packaging : zip uploadable ou déployable via Git
- Tests unitaires avec le framework de test DSS plugin
- Paramétrage : formulaires dynamiques définis en JSON

Code Environments (Code Envs)

- Environnements Python/R isolés par projet ou partagés
- Gestion des dépendances pip/conda indépendante du système
- Containérisé : chaque code env peut être un container Docker
- Essentiel pour la reproductibilité et la gestion des conflits de versions

9. Collaboration & Gouvernance

Permissions et sécurité

- Niveaux de permission : Admin, Reader, Writer, Manager par projet
- Groupes d'utilisateurs : LDAP/AD intégration pour gestion SSO
- Row-level security sur les datasets (filtrage par utilisateur)
- Column-level masking : masquage de colonnes sensibles
- Audit log : traçabilité de toutes les actions utilisateur
- Data catalog : inventaire et description des données accessibles

Versioning & Git

- Intégration Git native : chaque projet = un dépôt Git
- Commit, push, pull, merge depuis l'interface DSS
- Branches : travail parallèle sur une même base de code
- Diff visuel des changements sur les recettes et la configuration
- Déploiement "infra as code" : tout le projet est versionnable

Collaboration temps réel

- Commentaires sur les objets (datasets, recettes, modèles, scénarios)
- Tags et labels pour organiser les objets
- Activity feed : journal des modifications du projet
- Notifications : alertes in-app et email sur les changements

10. Administration DSS

Connexions (Connections)

- SQL : PostgreSQL, MySQL, Oracle, SQL Server, Teradata, Vertica, Greenplum, DB2
- Cloud data warehouses : Snowflake, BigQuery, Redshift, Synapse, Databricks
- Object storage : S3, Azure Blob (ADLS), GCS, OVH Object Storage
- HDFS / Hadoop : Cloudera, Hortonworks, EMR, HDInsight
- NoSQL : MongoDB, Cassandra, Elasticsearch, DynamoDB
- Messaging : Kafka, Kinesis
- SaaS : Salesforce, HubSpot, Google Analytics (via plugins)
- JDBC générique : tout driver JDBC

Gestion des clusters

- Cluster statique : cluster Spark/Hadoop toujours allumé, défini manuellement
- Cluster dynamique (EKS, GKE, AKS, EMR, Dataproc) : créé à la demande, détruit après job
- Kubernetes : déploiement de workloads dans des pods isolés
- Configuration Spark : memory, cores, shuffle partitions, adaptive query execution
- Définition de profils d'exécution : chaque recette peut utiliser un profil différent

Monitoring serveur

- Diagnostics intégrés : santé du serveur, espace disque, RAM, CPU
- Logs centralisés : backend.log, frontend.log, et logs par job
- Intégration Prometheus / Grafana pour monitoring externalisé
- API de status : endpoint REST pour supervision externe

11. Performance & Partitionnement

Partitionnement

Le partitionnement découpe un dataset en sous-ensembles logiques. C'est un levier clé pour traiter de gros volumes efficacement.

- Dimensions de partitionnement : date (year/month/day/hour), string discret, integer range
- Partitionnement temporel : très courant sur les données transactionnelles
- Chaînage : si A est partitionné, la recette A!B peut partitionner B avec la même logique
- Rebuild partiel : reconstruire seulement les partitions modifiées
- Dépendances entre partitions : ex. "construire le mois M en utilisant M-1"
- Propagation automatique dans le flow : DSS propage les schémas de partitionnement
- Bénéfices : réduction des temps de calcul, parallélisation, gestion des données fraîches

Optimisation in-database

- Préférer les recettes SQL sur des sources JDBC : pas de transfert de données
- Push-down computation : DSS génère du SQL natif pour la BDD cible
- Éviter de "matérialiser" des datasets intermédiaires inutiles dans DSS
- Utiliser les index et partitions de la base cible

Optimisation Spark

- Broadcast join pour les petites tables (spark.sql.autoBroadcastJoinThreshold)
- Ajuster le nombre de partitions Spark (spark.sql.shuffle.partitions)
- Activer l'Adaptive Query Execution (AQE) — Spark 3.x
- Persister les datasets intermédiaires coûteux
- Éviter les UDF Python sur Spark (préférer les fonctions natives)
- Utiliser Parquet/ORC comme format de stockage intermédiaire

12. Bonnes pratiques

Organisation du flow

- Zone Architecture : Discovery !' Preparation !' Modeling !' Serving
- Nommer les datasets avec un préfixe de zone : raw_, prep_, feat_, out_
- Limiter la profondeur du flow (max 7-8 niveaux) pour la lisibilité
- Utiliser les zones DSS pour compartimenter visuellement le flow
- Documenter les datasets et recettes avec des descriptions claires
- Toujours taguer les datasets "source of truth" vs "temporaires"

Code et reproductibilité

- Toujours utiliser des Code Environments — jamais le système Python
- Commiter régulièrement dans Git avec des messages de commit clairs
- Externaliser les paramètres (variables DSS, secrets vault) — pas de hardcoding
- Écrire des tests sur les données (métriques + checks) pour valider la qualité
- Préférer les recettes Visual quand elles suffisent (plus maintenables)
- Pour le ML : documenter les choix de features, algorithmes et métriques cibles

Production et MLOps

- Séparer les nœuds Design / Automation / API en production
- Versionner les modèles avant tout déploiement
- Mettre en place des checks de données à chaque étape critique
- Monitorer la dérive des données et des prédictions en production
- Prévoir un processus de rollback en cas de dégradation du modèle
- Documenter les modèles avec la génération de rapport DSS

13. Erreurs fréquentes

Erreurs de débutants

- Ne pas utiliser de Code Environment !' conflits de librairies, non-reproductibilité
- Oublier de commiter dans Git !' perte de travail, impossible de revenir en arrière
- Utiliser des chemins absolus dans le code !' problème lors du changement de serveur
- Hardcoder des credentials dans les recettes Python !' faille de sécurité majeure
- Matérialiser trop de datasets intermédiaires !' consommation de stockage excessive
- Ignorer les types de données dans le schéma !' erreurs silencieuses downstream

Erreurs de performance

- Utiliser le back-end DSS (Pandas) sur des volumes > 100M lignes !' OOM
- Ne pas partitionner les datasets temporels !' rebuild complet à chaque exécution
- Trop de shuffle Spark (jointures sur clés non partitionnées) !' temps de calcul élevé
- Charger tout un dataset Spark en mémoire Python (toPandas()) !' crash mémoire
- Ne pas configurer la taille des exécuteurs Spark !' sous-utilisation ou OOM
- Boucles Python sur des datasets Spark au lieu de transformations natives

Erreurs MLOps

- Déployer un modèle sans monitoring !' dérive non détectée
- Data leakage : features calculées avec des données futures !' métriques trop optimistes
- Ne pas versionner les modèles avant de redéployer !' impossible de rollback
- Séparer train/test sur des données temporelles sans tenir compte du temps
- Oublier de normaliser les features numériques pour les modèles linéaires/SVM
- Ne pas gérer les valeurs manquantes !' erreurs de scoring en production
- Évaluer uniquement sur AUC sans regarder la calibration !' mauvais score de probabilité

Erreurs d'administration

- Donner des droits Admin à tous les utilisateurs !' risque de sécurité
- Ne pas configurer de connexion partagée !' chaque user a ses propres credentials
- Ignorer l'espace disque sur le nœud DSS !' pannes en production
- Ne pas séparer les nœuds Design/Automation !' contention de ressources
- Oublier de configurer les backups du répertoire de données DSS

14. Éléments de pitch client

Problèmes résolus par Dataiku

- "Nos data scientists passent 80% de leur temps à préparer les données" !' Dataiku accélère la préparation x3 à x10
- "Les modèles ML restent dans des notebooks, jamais en production" !' MLOps intégré : déploiement en quelques clics
- "Les équipes data et métier ne se comprennent pas" !' Interface visuelle accessible aux non-développeurs
- "Impossible de tracer qui a fait quoi sur nos données" !' Audit log, Git, data lineage complet
- "Nos pipelines de données sont fragiles et non documentés" !' Flow visuel, métriques de qualité, scénarios

ROI et métriques clés

- Réduction du time-to-market des modèles ML : de 6-12 mois à 4-6 semaines
- Réduction du coût de maintenance des pipelines : moins de code custom à maintenir
- Collaboration : un seul outil pour data engineers, data scientists et analystes
- Gouvernance : conformité RGPD facilitée (lineage, masquage, audit)
- Scalabilité : du poste de travail au cluster Spark distribué, même interface

Objections courantes et réponses

- "C'est trop cher" !' Comparer le coût de licences vs coût de développement custom + maintenance
- "On peut faire ça avec Python/Jupyter" !' Gouvernance, collaboration, MLOps, partage : impossible sans plateforme
- "Notre SI est trop complexe" !' Connecteurs natifs pour 50+ sources, API REST pour le reste
- "On est déjà sur Azure ML / AWS Sagemaker" !' Dataiku peut s'intégrer ET orchestrer ces plateformes
- "Comment savoir que les modèles fonctionnent encore ?" !' Monitoring et alertes de dérive intégrés

15. Questions d'entretien technique

Questions fondamentales

- Quelle est la différence entre un dataset managed et un dataset external dans DSS ?
- Expliquez le concept de back-end d'exécution dans Dataiku. Quand utiliser Spark vs in-database ?
- Comment fonctionne le partitionnement dans DSS ? Donnez un exemple concret.
- Quelle est la différence entre une recette Python et une recette PySpark ?
- Comment gérez-vous les dépendances Python dans Dataiku en production ?

Questions MLOps

- Comment déployez-vous un modèle ML en production avec Dataiku ?
- Expliquez le concept de Champion/Challenger dans DSS. Comment l'implémentez-vous ?
- Comment détectez-vous la dérive d'un modèle en production ?
- Qu'est-ce que le Feature Store dans Dataiku et pourquoi l'utiliser ?
- Comment évitez-vous le data leakage lors de la création d'un modèle ML ?

Questions architecture/admin

- Quelle est la différence entre un Design Node, un Automation Node et un API Node ?
- Comment organisez-vous les permissions dans un projet multi-équipes ?
- Comment intégrez-vous Git dans un workflow DSS en production ?
- Quels sont les leviers de performance pour traiter 10 milliards de lignes dans DSS ?
- Comment configurez-vous un cluster Spark dynamique dans DSS ?

Questions scénarios et automatisation

- Comment construiriez-vous un pipeline de retraining automatique quotidien ?
- Comment chaînez-vous plusieurs scénarios dans DSS ?
- Quels types de triggers utilisez-vous pour déclencher un scénario en production ?
- Comment mettez-vous en place des alertes sur la qualité des données ?
- Comment orchestrez-vous Dataiku avec un outil externe (Airflow, Azure Data Factory) ?

